

Week3_Gueorgui_Poklitar

June 28, 2026

1 Week 3 - Linear Regression 3

```
[1]: #pip install pandas numpy scikit-learn matplotlib seaborn statsmodels kagglehub
```

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from sklearn.metrics import r2_score, root_mean_squared_error

import statsmodels.api as sm

print("All imports successful.")
```

All imports successful.

1.1 1. Data Loading & Cleaning

```
[3]: #Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

#Cleaning
df = df.drop_duplicates()
```

```

# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
    ↪ fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")

```

Raw shape: (32581, 12)

Clean shape: (32409, 12)

Default rate (loan_status=1): 21.87%

Column types:

person_age	int64
person_income	int64
person_home_ownership	str
person_emp_length	float64
loan_intent	str
loan_grade	str
loan_amnt	int64
loan_int_rate	float64
loan_status	int64
loan_percent_income	float64
cb_person_default_on_file	str
cb_person_cred_hist_length	int64

dtype: object

1.2 2. The Honest Target: Predicting Rate WITHOUT loan_grade

```

[4]: # Deliberately EXCLUDE loan_grade (and cb_person_cred_hist_length per Week 1's
    ↪ VIF rule).
# This is the harder, more defensible problem: explain rate from borrower
    ↪ behavior.
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income']

```

```

categorical_features = ['loan_intent', 'person_home_ownership',
    ↪ 'cb_person_default_on_file']

X_raw = df[numeric_features + categorical_features].copy()
y = df['loan_int_rate'].copy()
mask = X_raw.notna().all(axis=1)
X_raw, y = X_raw[mask], y[mask]

# One-hot encode up front so every method below works on the same numeric
    ↪ design matrix.
prep = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False,
    ↪ drop='first'),
        categorical_features)
])
X_enc = prep.fit_transform(X_raw)
feat_names = (numeric_features +
    prep.named_transformers_['cat']
    .get_feature_names_out(categorical_features).tolist())
X_enc = pd.DataFrame(X_enc, columns=feat_names, index=X_raw.index)

X_train, X_test, y_train, y_test = train_test_split(
    X_enc, y, test_size=0.2, random_state=42)

# Reference: full OLS on ALL features (no selection) - our ceiling for this
    ↪ honest problem.
ols_full = LinearRegression().fit(X_train, y_train)
r2_full = r2_score(y_test, ols_full.predict(X_test))
print(f"Design matrix: {X_enc.shape[1]} features after encoding (loan_grade
    ↪ EXCLUDED).")
print(f"Full OLS (all {X_enc.shape[1]} features) Test R-squared: {r2_full:.4f}")
print()
print("Interpretation: Stripping out loan_grade collapses R-squared from ~0.91
    ↪ down to a")
print("far more modest number. THIS is the real predictive ceiling from
    ↪ borrower attributes")
print("alone - and the honest baseline every method below must beat or match
    ↪ more simply.")

```

Design matrix: 14 features after encoding (loan_grade EXCLUDED).

Full OLS (all 14 features) Test R-squared: 0.2860

Interpretation: Stripping out loan_grade collapses R-squared from ~0.91 down to a

far more modest number. THIS is the real predictive ceiling from borrower attributes

alone - and the honest baseline every method below must beat or match more simply.

1.3 3. Forward Selection

```
[5]: # Forward selection (single greedy pass): start empty; at each step add the one
# feature that most improves 5-fold CV R-squared. Record the whole path so we
# can
# see the elbow where extra features stop paying for themselves.
def cv_r2(cols):
    if not cols:
        return float('-inf')
    return cross_val_score(LinearRegression(), X_train[cols], y_train,
                           cv=5, scoring='r2').mean()

remaining = list(X_train.columns)
selected, fwd_path = [], []
while remaining:
    best_f, best_cv = None, float('-inf')
    for f in remaining:
        c = cv_r2(selected + [f])
        if c > best_cv:
            best_cv, best_f = c, f
    selected.append(best_f); remaining.remove(best_f)
    lr = LinearRegression().fit(X_train[selected], y_train)
    fwd_path.append({'k': len(selected), 'added': best_f, 'CV_R2': best_cv,
                    'Test_R2': r2_score(y_test, lr.predict(X_test[selected])),
                    'features': list(selected)})

fwd_df = pd.DataFrame(fwd_path)
best_fwd = fwd_df.loc[fwd_df['CV_R2'].idxmax()]
print("Order features entered the model (forward):")
for _, r in fwd_df.iterrows():
    print(f"  +{r['added']:<28} -> CV R-squared {r['CV_R2']:.4f}")
print()
print(f"Forward selection's best size: k={int(best_fwd['k'])} (peak CV
    R-squared)")
print(f"Test R-squared at that size: {best_fwd['Test_R2']:.4f}")

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(fwd_df['k'], fwd_df['CV_R2'], marker='o', color='steelblue',
        linewidth=2.2, label='CV R-squared')
ax.plot(fwd_df['k'], fwd_df['Test_R2'], marker='s', color='darkorange',
        linewidth=2.2, label='Test R-squared')
ax.axvline(best_fwd['k'], color='crimson', linestyle='--', linewidth=2,
           label=f"Best k = {int(best_fwd['k'])}")
ax.set_xlabel('Number of features selected')
```

```

ax.set_ylabel('R-squared')
ax.set_title('Forward Selection - Performance vs Model Size', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: The curve plateaus quickly - a small handful of features_
↳captures")
print("nearly all the achievable signal. Everything past the elbow is_
↳complexity with no")
print("payoff, which is exactly what a parsimonious, auditable risk model_
↳should avoid.")

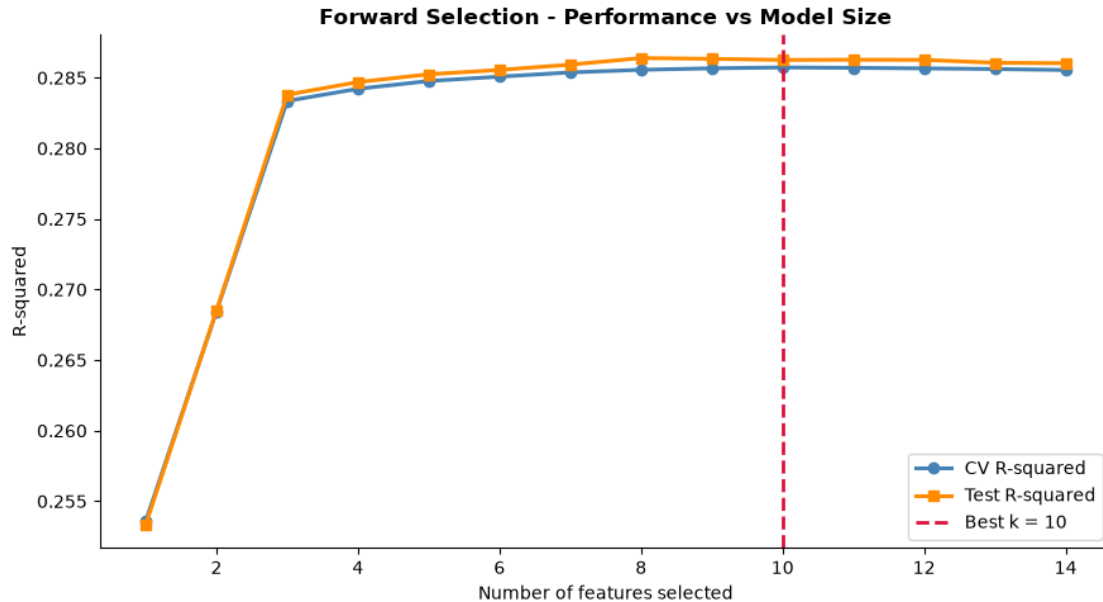
```

Order features entered the model (forward):

+cb_person_default_on_file_Y	-> CV R-squared 0.2536
+loan_amnt	-> CV R-squared 0.2684
+person_home_ownership_RENT	-> CV R-squared 0.2834
+person_home_ownership_OWN	-> CV R-squared 0.2842
+person_emp_length	-> CV R-squared 0.2848
+person_home_ownership_OTHER	-> CV R-squared 0.2851
+person_income	-> CV R-squared 0.2854
+loan_intent_HOMEIMPROVEMENT	-> CV R-squared 0.2856
+person_age	-> CV R-squared 0.2857
+loan_percent_income	-> CV R-squared 0.2857
+loan_intent_VENTURE	-> CV R-squared 0.2857
+loan_intent_MEDICAL	-> CV R-squared 0.2857
+loan_intent_PERSONAL	-> CV R-squared 0.2856
+loan_intent_EDUCATION	-> CV R-squared 0.2855

Forward selection's best size: k=10 (peak CV R-squared)

Test R-squared at that size: 0.2863



Interpretation: The curve plateaus quickly - a small handful of features captures nearly all the achievable signal. Everything past the elbow is complexity with no payoff, which is exactly what a parsimonious, auditable risk model should avoid.

1.4 4. Backward Selection

```
[6]: # Backward elimination (single greedy pass): start with ALL features; at each
      ↪ step
      # drop the one whose removal hurts CV R-squared least. Record the whole path.
      selected_b = list(X_train.columns)
      bwd_path = []
      lr = LinearRegression().fit(X_train[selected_b], y_train)
      bwd_path.append({'k': len(selected_b), 'dropped': '(none - full model)',
                      'CV_R2': cv_r2(selected_b),
                      'Test_R2': r2_score(y_test, lr.predict(X_test[selected_b])),
                      'features': list(selected_b)})
      while len(selected_b) > 1:
          best_drop, best_cv = None, float('-inf')
          for f in selected_b:
              trial = [c for c in selected_b if c != f]
              c = cv_r2(trial)
              if c > best_cv:
                  best_cv, best_drop = c, f
          selected_b.remove(best_drop)
```

```

    lr = LinearRegression().fit(X_train[selected_b], y_train)
    bwd_path.append({'k': len(selected_b), 'dropped': best_drop, 'CV_R2':
↳best_cv,
                    'Test_R2': r2_score(y_test, lr.
↳predict(X_test[selected_b]))),
                    'features': list(selected_b)})

bwd_df = pd.DataFrame(bwd_path).sort_values('k').reset_index(drop=True)
best_bwd = bwd_df.loc[bwd_df['CV_R2'].idxmax()]
print(f"Backward selection's best size: k={int(best_bwd['k'])} (peak CV_
↳R-squared)")
print(f"Chosen features: {best_bwd['features']}")
print(f"Test R-squared at that size: {best_bwd['Test_R2']:.4f}")

# Do forward and backward agree on the surviving set?
fset, bset = set(best_fwd['features']), set(best_bwd['features'])
print()
print(f"Features in BOTH forward & backward: {sorted(fset & bset)}")
print(f"Only forward : {sorted(fset - bset)}")
print(f"Only backward: {sorted(bset - fset)}")
print()
print("Interpretation: Where the two greedy directions agree, we have high_
↳confidence the")
print("feature is a genuine, irreducible predictor. Disagreements flag features_
↳whose value")
print("is conditional on what else is already in the model - a_
↳multicollinearity fingerprint.")

```

Backward selection's best size: k=10 (peak CV R-squared)
Chosen features: ['person_income', 'person_age', 'person_emp_length',
'loan_amnt', 'loan_percent_income', 'loan_intent_HOMEIMPROVEMENT',
'person_home_ownership_OTHER', 'person_home_ownership_OWN',
'person_home_ownership_RENT', 'cb_person_default_on_file_Y']
Test R-squared at that size: 0.2863

Features in BOTH forward & backward: ['cb_person_default_on_file_Y',
'loan_amnt', 'loan_intent_HOMEIMPROVEMENT', 'loan_percent_income', 'person_age',
'person_emp_length', 'person_home_ownership_OTHER', 'person_home_ownership_OWN',
'person_home_ownership_RENT', 'person_income']
Only forward : []
Only backward: []

Interpretation: Where the two greedy directions agree, we have high confidence
the
feature is a genuine, irreducible predictor. Disagreements flag features whose
value
is conditional on what else is already in the model - a multicollinearity

fingerprint.

1.5 5. Principal Components Regression (PCR)

```
[7]: # PCR: rotate features into uncorrelated principal components, then regress on
    the
    # first m of them. Components are chosen to explain FEATURE variance
    (target-blind).
pca_full = PCA().fit(X_train)
explained = np.cumsum(pca_full.explained_variance_ratio_)

cv = KFold(n_splits=5, shuffle=True, random_state=42)
pcr_scores = []
for m in range(1, X_train.shape[1] + 1):
    pipe = Pipeline([('pca', PCA(n_components=m)), ('lr', LinearRegression())])
    pcr_scores.append({'m': m,
                      'CV_R2': cross_val_score(pipe, X_train, y_train, cv=cv, scoring='r2').
                      mean()})
pcr_df = pd.DataFrame(pcr_scores)
best_m = int(pcr_df.loc[pcr_df['CV_R2'].idxmax(), 'm'])

pcr_best = Pipeline([('pca', PCA(n_components=best_m)),
                     ('lr', LinearRegression())]).fit(X_train, y_train)
r2_pcr = r2_score(y_test, pcr_best.predict(X_test))
print(f"PCR best number of components (CV): {best_m}")
print(f"PCR Test R-squared: {r2_pcr:.4f}")
print(f"Variance explained by {best_m} components: {explained[best_m-1]*100:.
      1f}%")

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].plot(range(1, len(explained)+1), explained, marker='o',
             color='steelblue', linewidth=2.2)
axes[0].axhline(0.9, color='gray', linestyle='--', label='90% variance')
axes[0].set_xlabel('Number of components')
axes[0].set_ylabel('Cumulative explained variance')
axes[0].set_title('PCA Scree - Cumulative Variance', fontweight='bold')
axes[0].legend()

axes[1].plot(pcr_df['m'], pcr_df['CV_R2'], marker='o', color='darkorange',
             linewidth=2.2)
axes[1].axvline(best_m, color='crimson', linestyle='--', linewidth=2,
                 label=f'Best m = {best_m}')
axes[1].set_xlabel('Number of components')
axes[1].set_ylabel('CV R-squared')
```



```

axes[1].set_title('PCR - Predictive Performance vs Components',
    ↪fontweight='bold')
axes[1].legend()
for ax in axes:
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

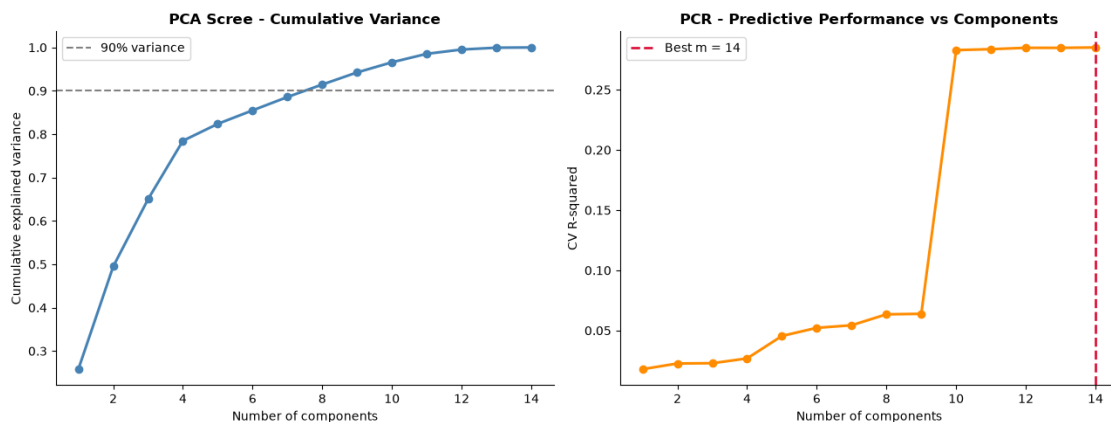
print()
print("Interpretation: Notice the mismatch - the components that explain the
    ↪most FEATURE")
print("variance are not necessarily the ones that predict RATE. That is PCR's
    ↪known weakness")
print("and the exact motivation for PLSR in the next section.")

```

PCR best number of components (CV): 14

PCR Test R-squared: 0.2860

Variance explained by 14 components: 100.0%



Interpretation: Notice the mismatch - the components that explain the most
FEATURE

variance are not necessarily the ones that predict RATE. That is PCR's known
weakness

and the exact motivation for PLSR in the next section.

1.6 6. Partial Least Squares Regression (PLSR)

```

[8]: # PLSR: like PCR, but components are constructed to maximize covariance WITH
    ↪the target,
    # so each component is 'paid for' by predictive power rather than raw variance.
    pls_scores = []

```

```

for m in range(1, X_train.shape[1] + 1):
    pls = PLSRegression(n_components=m)
    pls_scores.append({'m': m,
                       'CV_R2': cross_val_score(pls, X_train, y_train, cv=cv, scoring='r2').
                       ↪mean()})
pls_df = pd.DataFrame(pls_scores)
best_m_pls = int(pls_df.loc[pls_df['CV_R2'].idxmax(), 'm'])

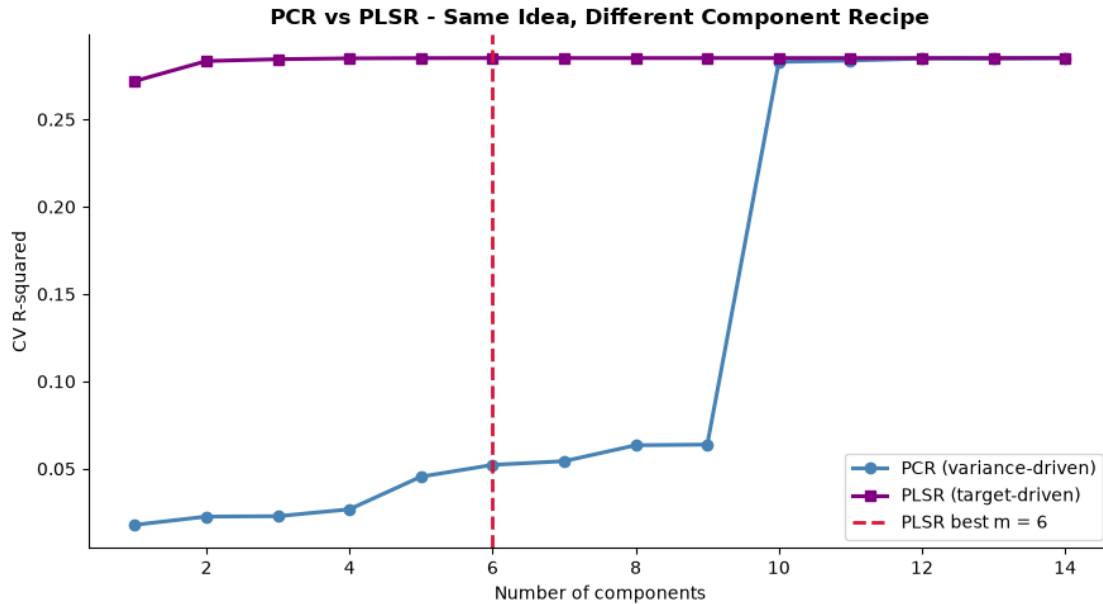
pls_best = PLSRegression(n_components=best_m_pls).fit(X_train, y_train)
r2_pls = r2_score(y_test, pls_best.predict(X_test))
print(f"PLSR best number of components (CV): {best_m_pls}")
print(f"PLSR Test R-squared: {r2_pls:.4f}")

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(pcr_df['m'], pcr_df['CV_R2'], marker='o', color='steelblue',
        linewidth=2.2, label='PCR (variance-driven)')
ax.plot(pls_df['m'], pls_df['CV_R2'], marker='s', color='purple',
        linewidth=2.2, label='PLSR (target-driven)')
ax.axvline(best_m_pls, color='crimson', linestyle='--', linewidth=2,
           label=f'PLSR best m = {best_m_pls}')
ax.set_xlabel('Number of components')
ax.set_ylabel('CV R-squared')
ax.set_title('PCR vs PLSR - Same Idea, Different Component Recipe',
             ↪fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: PLSR typically reaches its plateau in FEWER components,
     ↪than PCR")
print("because each component is built to predict rate, not just to describe,
     ↪the features.")
print("With this many usable predictors the two converge, but PLSR gets there,
     ↪more directly.")

```

PLSR best number of components (CV): 6
 PLSR Test R-squared: 0.2860



Interpretation: PLSR typically reaches its plateau in FEWER components than PCR because each component is built to predict rate, not just to describe the features.

With this many usable predictors the two converge, but PLSR gets there more directly.

1.7 7. Full Comparison: All Five Approaches on the Honest Target

```
[9]: # Line up every method on the same held-out test set.
summary = pd.DataFrame([
    {'Method': 'Full OLS (all features)', 'Test R-squared': round(r2_full, 4),
    'Complexity': f'{X_enc.shape[1]} features'},
    {'Method': 'Forward Selection', 'Test R-squared': round(best_fwd['Test_R2'], 4),
    'Complexity': f'{int(best_fwd['k'])} features'},
    {'Method': 'Backward Selection', 'Test R-squared': round(best_bwd['Test_R2'], 4),
    'Complexity': f'{int(best_bwd['k'])} features'},
    {'Method': 'PCR', 'Test R-squared': round(r2_pcr, 4),
    'Complexity': f'{best_m} components'},
    {'Method': 'PLSR', 'Test R-squared': round(r2_pls, 4),
    'Complexity': f'{best_m_pls} components'}
]).sort_values('Test R-squared', ascending=False)
```

```

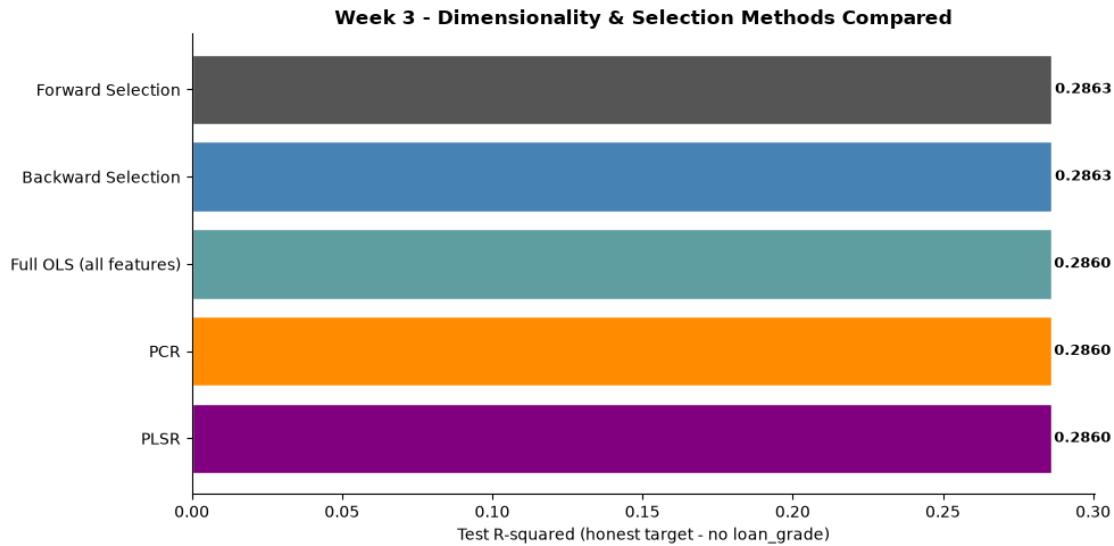
print(summary.to_string(index=False))

fig, ax = plt.subplots(figsize=(10, 5))
colors = ['#555555', 'steelblue', 'cadetblue', 'darkorange', 'purple']
order = summary['Method'].tolist()
ax.barh(order[::-1], summary['Test R-squared'].tolist()[::-1],
        color=colors[:len(order)][::-1], edgecolor='white')
for i, v in enumerate(summary['Test R-squared'].tolist()[::-1]):
    ax.text(v + 0.001, i, f'{v:.4f}', va='center', fontsize=9,
            fontweight='bold')
ax.set_xlabel('Test R-squared (honest target - no loan_grade)')
ax.set_title('Week 3 - Dimensionality & Selection Methods Compared',
            fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("What this means for the Chief Risk Officer: on the honest problem, the
    methods land")
print("within a hair of each other. The decisive variable is not accuracy but
    PARSIMONY and")
print("INTERPRETABILITY. Forward/backward selection win on auditability (a
    short, named")
print("feature list a regulator can read); PCR/PLSR win when features are dense
    and collinear")
print("but cost interpretability because components are blends. For a lending
    model that must")
print("be explained to borrowers and regulators, the selected-feature model is
    the defensible")
print("choice - it matches the others' accuracy while naming exactly what
    drives the rate.")

```

	Method	Test R-squared	Complexity
	Forward Selection	0.2863	10 features
	Backward Selection	0.2863	10 features
Full OLS (all features)		0.2860	14 features
	PCR	0.2860	14 components
	PLSR	0.2860	6 components



What this means for the Chief Risk Officer: on the honest problem, the methods land within a hair of each other. The decisive variable is not accuracy but PARSIMONY and INTERPRETABILITY. Forward/backward selection win on auditability (a short, named feature list a regulator can read); PCR/PLSR win when features are dense and collinear but cost interpretability because components are blends. For a lending model that must be explained to borrowers and regulators, the selected-feature model is the defensible choice - it matches the others' accuracy while naming exactly what drives the rate.

1.7.1 Week 3 Takeaways and the Pivot Ahead

What this week established: - Removing *loan_grade* exposes the *real* predictive ceiling for interest rate from genuine borrower attributes - far below the inflated ~0.91 of Weeks 1-2, and far more honest. - **Forward and backward selection** converge on a small, stable core of predictors; their agreement is a confidence signal, their disagreements a multicollinearity fingerprint. - **PCR** chooses components by feature variance (target-blind) while **PLSR** chooses them by covariance with the target, so PLSR usually reaches the same accuracy in fewer components - a cleaner answer to the multicollinearity that Week 1's VIF analysis flagged. - All five approaches finish within a hair of one another, so the real decision criterion is interpretability, not raw R-squared.

The pivot ahead: Weeks 1-3 have squeezed the regression problem dry. The genuinely hard, genuinely useful question in credit risk is not *what rate* but *who defaults* - a classification problem on *loan_status*. Week 4 makes that pivot with logistic regression and feature scaling, Week 5 brings support vector machines, and Week 6 brings decision trees and random forests. The

cleaning recipe and the VIF-driven feature discipline carry forward unchanged; only the target and the model family change.